



Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 5

Belo Horizonte
9 de junho de 2026

SUMÁRIO

1	EXERCÍCIO 9.2	3
2	EXERCÍCIO 9.3	6
2.1	Item (a)	6
2.2	Item (b)	8
3	EXERCÍCIO 9.4	9
3.1	Item (a)	9
3.2	Item (b)	9
4	EXERCÍCIO 9.8	12
ANEXO A	CÓDIGO EXERCÍCIO 9.2	14
ANEXO B	CÓDIGO EXERCÍCIO 9.3	16
ANEXO C	CÓDIGO EXERCÍCIO 9.4	19
ANEXO D	CÓDIGO EXERCÍCIO 9.8	21

1 EXERCÍCIO 9.2

Todo o código usado nesse exercício encontra-se no Anexo A.

Considerando o problema do exercício 8.2 em que temos um domínio 2D quadrado e queremos simular o modo TM_z com o FDTD, usamos os seguintes parâmetros de discretização:

- $N_x = 200$, $N_y = 200$
- $\Delta y = \Delta x = 1$ mm
- $\Delta t = \frac{0.9}{c\sqrt{(\Delta x)^2 + (\Delta y)^2}} = 2.12$ ps (condição CFL 2D)
- $N_t = 800$

A fonte aplicada no centro do domínio é uma função gaussiana dada por

$$f(t) = e^{-\frac{1}{2}\left(\frac{n-40}{12}\right)^2}$$

que tem um pico em $t = 84$ ps e uma largura de aproximadamente 25 ps.

Os parâmetros da CPML adotados foram os seguintes:

- $N_{pml} = 30$, $m = 3$
- $R_{err} = 10^{-6}$
- $\eta = \sqrt{\frac{\mu}{\epsilon}}$
- $\sigma_{max} = -\frac{(m+1)\ln(R_{err})}{2\eta N_{pml}} = 2.44$

A propagação obtida está exibida na Figura 1, onde podemos observar que o campo se propaga para fora do domínio sem reflexões significativas, indicando que os PMLs estão funcionando corretamente. Com uma ponta de prova próximo ao centro do grid, verificamos uma leve reflexão do campo, como mostra a Figura 2.

O coeficiente de reflexão pode ser calculado a partir dos picos verificados na Figura 2, obtendo:

$$R = \frac{0.00007}{0.01} = 0.007 = 0.7\%$$

o qual, quando comparado com a reflexão obtida usando o Mur de $R = 5\%$ no exercício 8.2, representa uma redução quase 10 vezes melhor usando a PML.

A Figura 3 mostra a evolução do campo E_z ao longo do tempo, em um corte horizontal no eixo x passando pelo centro da cavidade. As linhas verticais vermelhas indicam a região onde o campo é absorvido pelos PMLs, e podemos observar que o campo decai para zero em todos os instantes, evidenciando a absorção da PML.

Alterando o ângulo θ da ponta de prova em relação ao centro do grid, podemos verificar como a reflexão da PML varia em relação ao ângulo de incidência. Variando

Figura 1 – Propagação de E_z ao longo do grid, sem reflexão nas bordas.

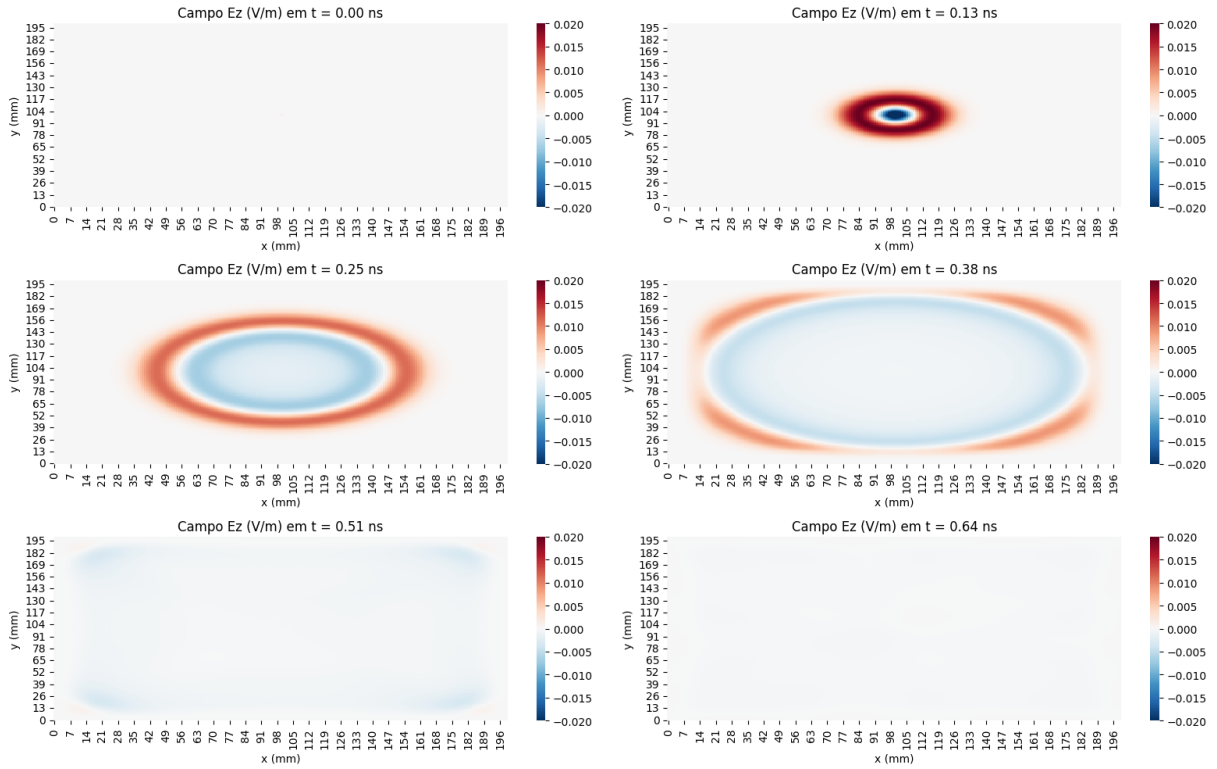


Figura 2 – Reflexão de E_z no centro do grid em função do tempo.

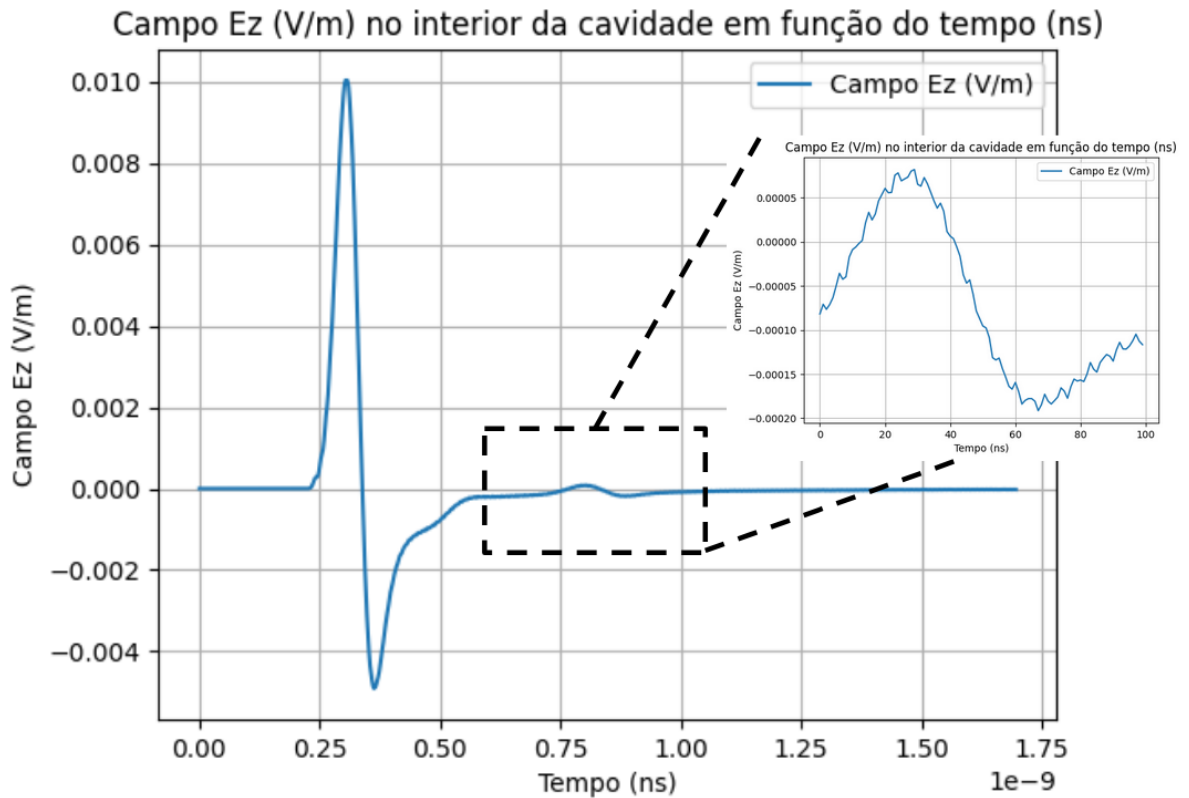
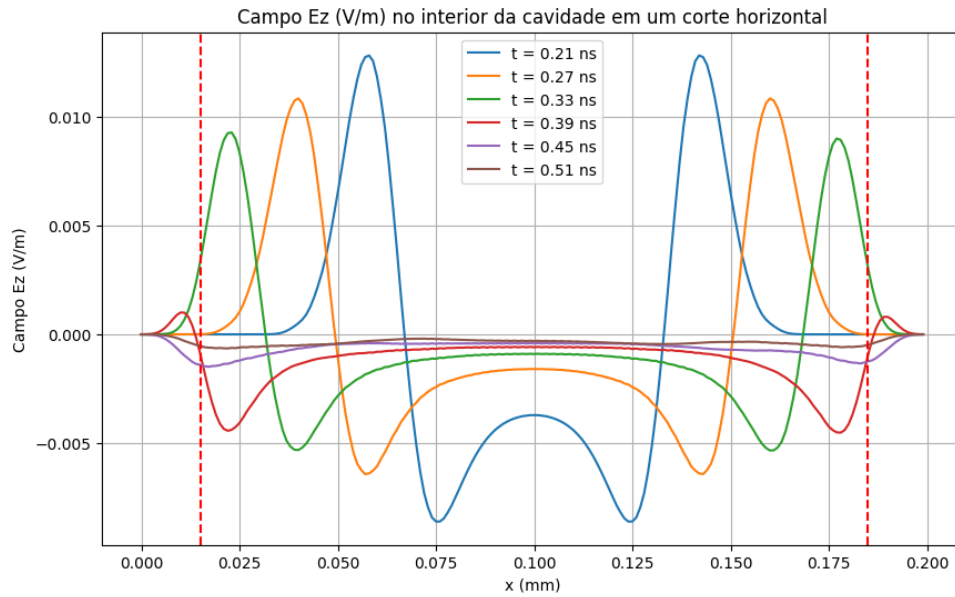


Figura 3 – Decaimento do campo E_z ao longo do eixo x ao entrar nas regiões de PML.



as posições p_x e p_y da ponta de prova, tendo em mente que o ponto $(N_x/2, N_y/2)$ corresponde ao centro do grid, obtemos os seguintes resultados para o coeficiente de reflexão R , exibidos na Tabela 1:

Tabela 1 – Variação do coeficiente de reflexão com o ângulo de incidência.

p_x	p_y	θ	R
$N_x/2 + 0$	$N_y/2 + 50$	0°	0.013
$N_x/2 + 50$	$N_y/2 + 30$	30°	0.016
$N_x/2 + 50$	$N_y/2 + 50$	45°	0.019
$N_x/2 + 30$	$N_y/2 + 50$	60°	0.016
$N_x/2 + 50$	$N_y/2 + 0$	90°	0.013

Vemos que a reflexão é máxima quando a onda incidente está inclinada em 45 graus em relação à normal da borda, e é mínima quando a onda incide perpendicularmente (0 ou 90 graus). Assim, temos um desempenho pior da PML nas quinas do domínio, conforme esperado.

2 EXERCÍCIO 9.3

Todo o código usado nesse exercício encontra-se no Anexo B.

Usamos os seguintes parâmetros de discretização:

- $N_x = 120$, $N_y = 60$
- $\Delta y = \Delta x = 5$ cm
- $\Delta t = \frac{0.9}{c\sqrt{(\Delta x)^2 + (\Delta y)^2}} = 0.1$ ns (condição CFL 2D)
- $N_t = 800$

A fonte aplicada no centro do domínio é uma função senoidal dada por

$$f(n) = \sin(2\pi f_0 n \Delta t)$$

com $f_0 = 600$ MHz.

Os parâmetros da CPML adotados foram os seguintes:

- $N_{pml} = 30$, $m = 4$
- $R_{err} = 10^{-6}$
- $\eta = \sqrt{\frac{\mu}{\epsilon}}$
- $\sigma_{max} = -\frac{(m+1)\ln(R_{err})}{2\eta N_{pml}} = 2.44$

A propagação obtida está exibida na Figura 4, onde podemos observar que o campo se propaga para fora do domínio sem reflexões significativas, indicando que os PMLs estão funcionando corretamente.

Olhando em um corte horizontal no eixo x passando pelo centro da cavidade, podemos observar o decaimento do campo E_z ao entrar nas regiões de PML, como mostra a Figura 5.

2.1 Item (a)

Para calcular o coeficiente de reflexão, utilizamos o seguinte procedimento:

1. Realiza a simulação completa com PML e coleta os pontos em uma ponta de prova perto do centro. Esse é o campo total E_{total}
2. Repete a simulação com as bordas muito longe (sem reflexão) e coleta os pontos em uma ponta de prova perto do centro. Esse é o campo incidente E_{inc}
3. Calcula o campo refletido como $E_{ref} = E_{total} - E_{inc}$
4. Calcula o coeficiente de reflexão como $R = \frac{|E_{ref}|}{|E_{inc}|}$, usando decibéis (dB)

Variando o valor do parâmetro m da PML, obtemos os valores exibidos na Tabela 2. Verificamos que o coeficiente de reflexão diminui à medida que aumentamos o valor de m , indicando uma melhor absorção da PML.

Figura 4 – Propagação de E_z ao longo do grid, sem reflexão nas bordas.

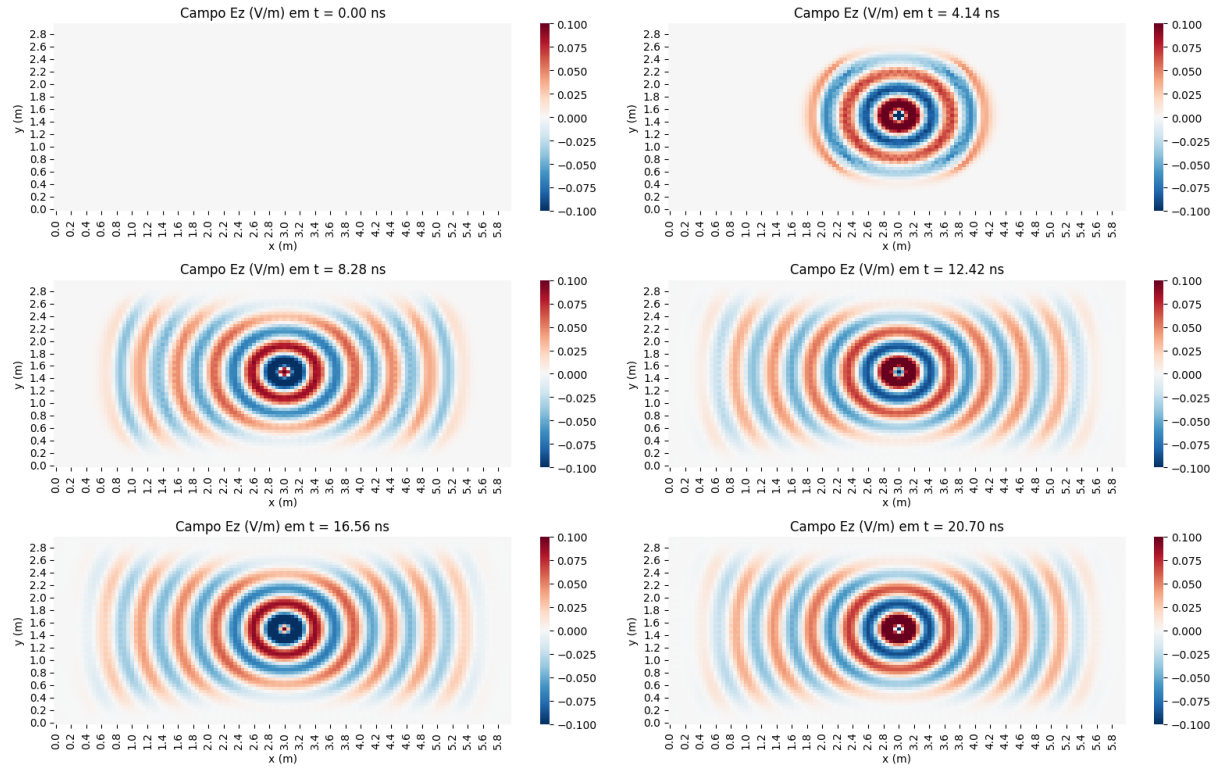


Figura 5 – Decaimento do campo E_z ao longo do eixo x ao entrar nas regiões de PML.

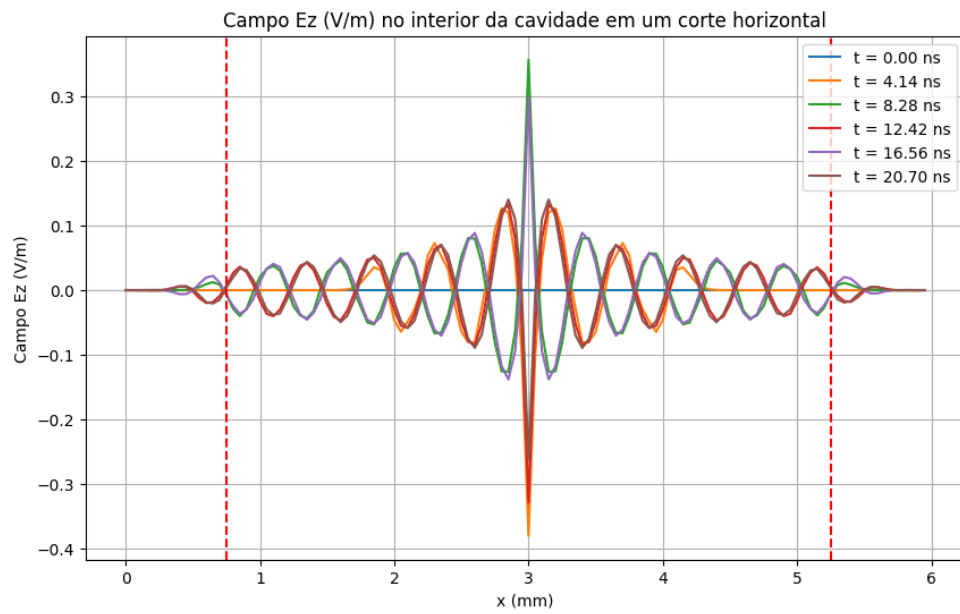


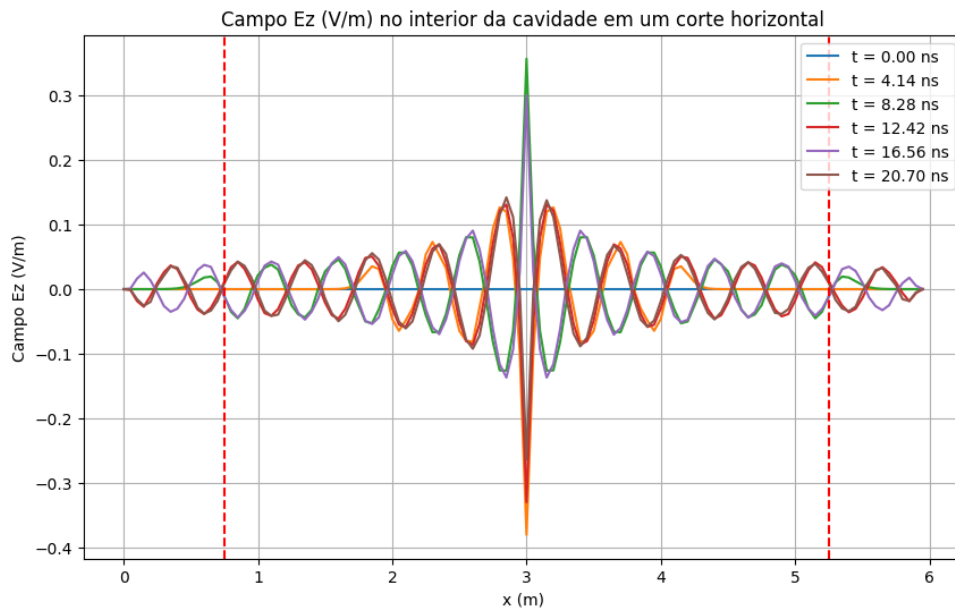
Tabela 2 – Variação do coeficiente de reflexão em função de m .

m	R (dB)
3	-22.21
3.25	-24.53
3.5	-26.33
3.75	-28.39
4	-30.61

2.2 Item (b)

Implementando o grading geométrico para o perfil de $\sigma(x)$, obtemos o perfil de decaimento exibido ao longo do eixo x exibido na Figura 6 obtemos os seguintes valores para o coeficiente de reflexão, exibidos na Tabela 3:

Figura 6 – Decaimento do campo E_z ao longo do eixo x ao entrar nas regiões de PML, com função σ com grading geométrico.

Tabela 3 – Variação do coeficiente de reflexão em função de g .

g	R (dB)
2.5	-51.28
3.0	-52.71
3.5	-52.98

Verificamos que o coeficiente de reflexão é significativamente menor usando o grading geométrico do que quando usamos o grading polinomial, indicando uma melhor absorção da PML. Além disso, o coeficiente se reduz ligeiramente ao aumentar o parâmetro g , mas a diferença não é tão grande quanto a variação do parâmetro m no grading polinomial.

3 EXERCÍCIO 9.4

Todo o código usado nesse exercício encontra-se no Anexo C.

Usamos os seguintes parâmetros de discretização (mesmos do exercício 9.3):

- $N_x = 120$, $N_y = 60$
- $\Delta y = \Delta x = 5$ cm
- $\Delta t = \frac{0.9}{c\sqrt{(\Delta x)^2 + (\Delta y)^2}} = 0.1$ ns (condição CFL 2D)
- $N_t = 800$

A fonte aplicada no centro do domínio é uma função senoidal dada por

$$f(n) = \sin(2\pi f_0 n \Delta t)$$

com $f_0 = 600$ MHz.

3.1 Item (a)

Os parâmetros da UPML adotados foram os seguintes:

- $N_{pml} = 30$, $m = 3.75$
- $R_{err} = 10^{-8}$
- $\sigma_{max} = -\frac{(m+1)\ln(R_{err})}{2N_{pml} \cdot 377} = 0.116$

A propagação obtida está exibida na Figura 7, onde podemos observar que o campo se propaga para fora do domínio sem reflexões significativas, indicando que os PMLs estão funcionando corretamente.

Olhando em um corte horizontal no eixo x passando pelo centro da cavidade, podemos observar o decaimento do campo E_z ao entrar nas regiões de PML, como mostra a Figura 8.

Usando o mesmo procedimento do exercício 9.3, obtemos um coeficiente de reflexão R de -33.67 dB, que é próximo do valor obtido no exercício anterior com a CPML.

3.2 Item (b)

Com $f_0 = 100$ MHz obtemos a propagação exibida na Figura 9, e o novo coeficiente de reflexão aumenta consideravelmente para - 17.6 dB, quase uma ordem de grandeza maior que $f_0 = 600$ MHz, indicando que a UPML não possui um bom desempenho se estiver a poucos comprimentos de onda de distância da fonte.

Figura 7 – Propagação de E_z ao longo do grid, sem reflexão nas bordas.

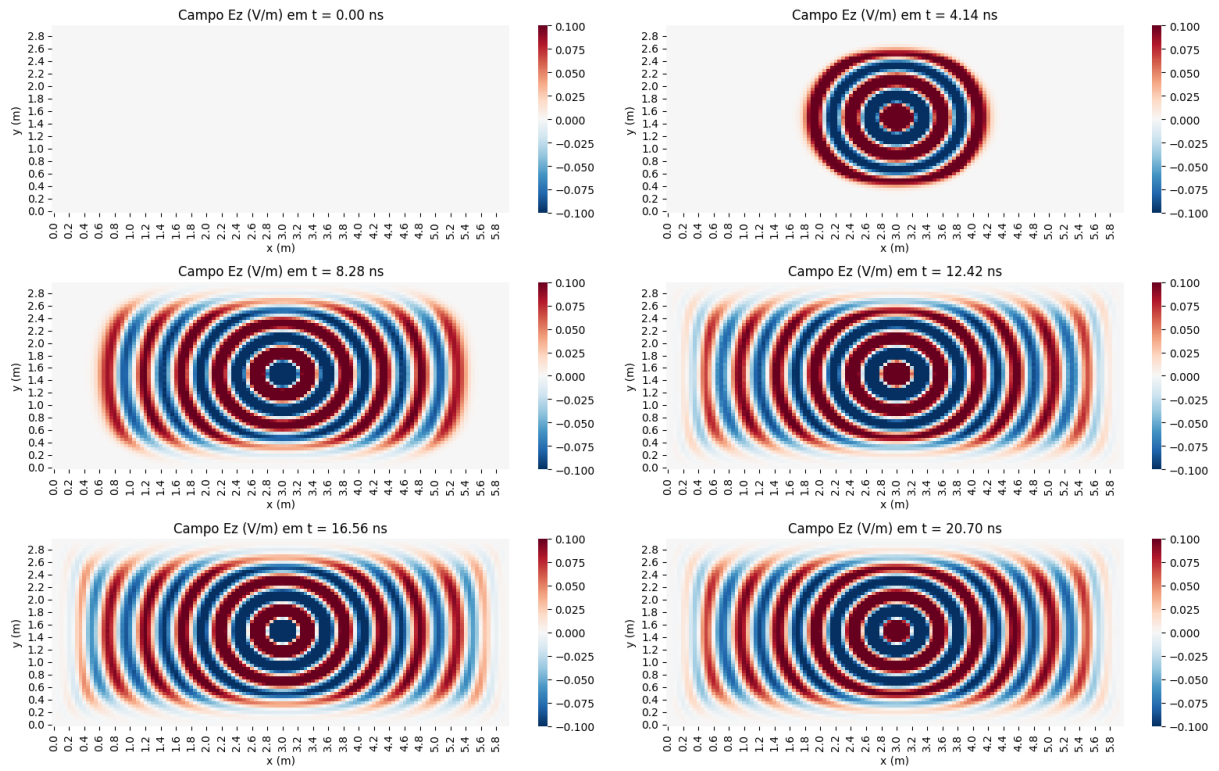


Figura 8 – Decaimento do campo E_z ao longo do eixo x ao entrar nas regiões de PML.

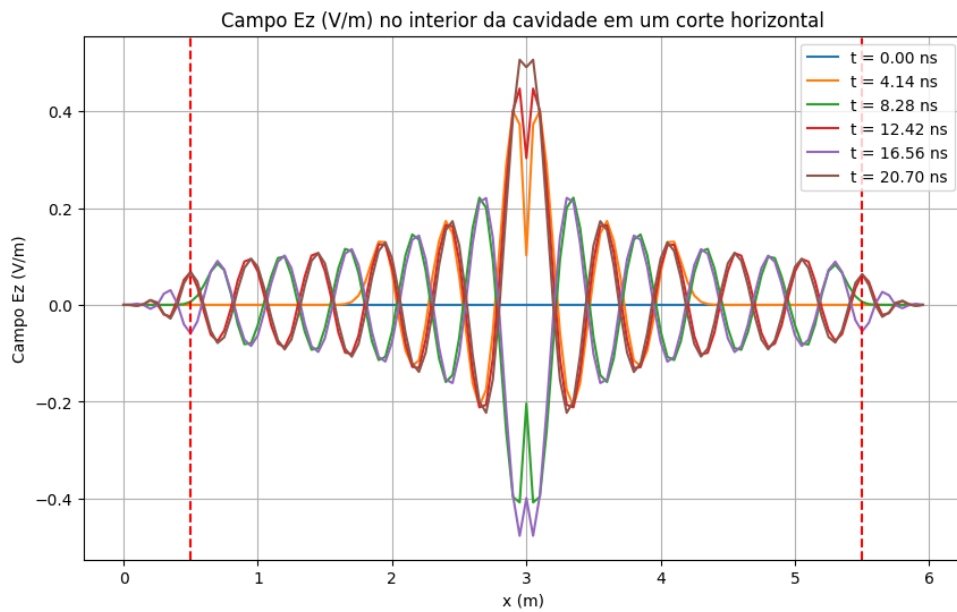
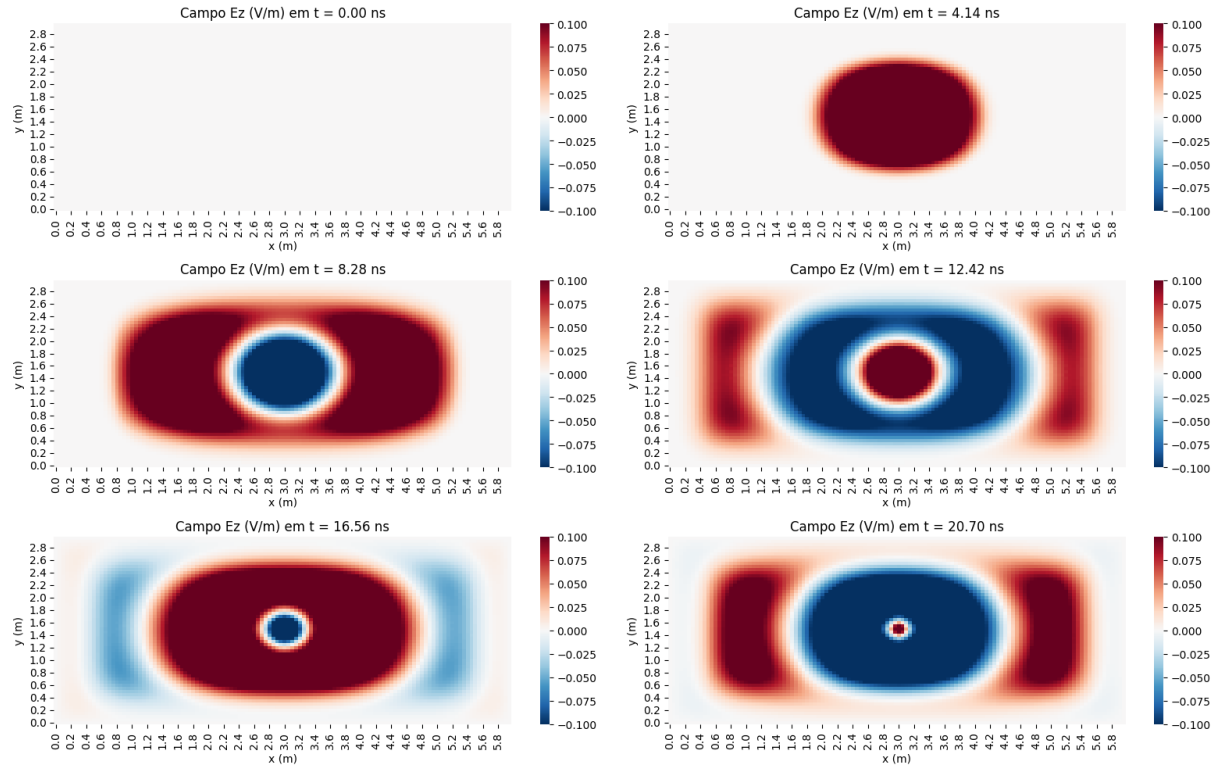


Figura 9 – Propagação de E_z ao longo do grid para $f_0 = 100$ MHz.



4 EXERCÍCIO 9.8

Todo o código usado nesse exercício encontra-se no Anexo D.

Usamos os seguintes parâmetros de discretização (mesmos do exercício 9.3):

- $N_x = 200$, $N_y = 200$
- $\Delta y = \Delta x = 1 \text{ mm}$
- $\Delta t = \frac{0.9}{c\sqrt{(\Delta x)^2 + (\Delta y)^2}} = 2.12 \text{ ps}$ (condição CFL 2D)
- $N_t = 800$

A fonte aplicada no centro do domínio é uma função gaussiana dada por

$$f(t) = e^{-\frac{1}{2}\left(\frac{n-40}{S}\right)^2}$$

onde S é um parâmetro que controla a abertura do pulso, estando relacionada com a frequência do sinal.

As progações obtidas com a UPML e CPML são semelhantes com os resultados obtidos nos exercícios anteriores. O objetivo nesse exercício é variar o parâmetro S e a localização da ponta de prova para verificar o desempenho da UPML e da CPML em função da frequência e do ângulo de incidência.

Como o pulso é gaussiano, para calcular o coeficiente de reflexão basta dividir a amplitude da onda incidente pela onda que foi refletida, como visto na Figura 2.

A Tabela 4 sumariza o resultado obtido nesse exercício.

Tabela 4 – Desempenho da UPML e CPML em função da frequência do sinal da fonte e do ângulo de incidência.

S	θ	$R_{CPML} \text{ (dB)}$	$R_{UPML} \text{ (dB)}$
$3 \cdot \Delta t$	0	-54.23	-41.64
$3 \cdot \Delta t$	15	-53.38	-41.77
$3 \cdot \Delta t$	30	-53.4	-42.65
$3 \cdot \Delta t$	45	-50.74	-43.83
$5 \cdot \Delta t$	0	-48.14	-35.98
$5 \cdot \Delta t$	15	-47.33	-35.38
$5 \cdot \Delta t$	30	-46.04	-36.09
$5 \cdot \Delta t$	45	-44.3	-37.23
$7 \cdot \Delta t$	0	-43.93	-31.35
$7 \cdot \Delta t$	15	-43.10	-31.38
$7 \cdot \Delta t$	30	-41.87	-31.86
$7 \cdot \Delta t$	45	-40.07	-33.06

Em relação a frequência, vemos que tanto a CPML quanto a UPML apresentam melhor desempenho em altas frequências do que em baixas. Em todos os casos, contudo, o desempenho da CPML foi melhor do que o da UPML, apresentando um coeficiente de reflexão cerca de 10 dB menor.

Em relação ao ângulo de incidência, vemos que a CPML apresenta melhor desempenho quando $\theta = 0$, e conforme o ângulo aumenta, a CPML apresenta maior reflexão do sinal. Contudo, a UPML apresenta o comportamento inverso, sendo que era esperado que ela atenuasse melhor os sinais incidentes em $\theta = 0$. De qualquer forma, a CPML também apresenta uma melhor reflexão que a UPML para todos os ângulos de incidência.

ANEXO A – CÓDIGO EXERCÍCIO 9.2

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

def compute_sigma(N, ds, n_pml, sigma_max, m_order):
    sig = np.zeros(N)
    for i in range(N):
        dist_left = (n_pml - i) * ds
        dist_right = (i - (N - 1 - n_pml)) * ds
        if dist_left > 0:
            sig[i] = sigma_max * (dist_left / (n_pml * ds))**m_order
        elif dist_right > 0:
            sig[i] = sigma_max * (dist_right / (n_pml * ds))**m_order
    return sig

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
Nx = Ny = 200
dx = dy = 1e-3
dt = 0.9 / (vp * np.sqrt(1/dx**2 + 1/dy**2))
Nt = 800
L = Nx * dx
T = Nt * dt

# --- PML ----
npml = 30
m = 3
R_err = 1e-6
eta = np.sqrt(mu / eps)
d_pml = npml * dx
sig_max = -(m + 1) * np.log(R_err) / (2 * eta * d_pml)

# Declara os campos
Ez = np.zeros((Nx, Ny))
Hx = np.zeros((Nx, Ny))
Hy = np.zeros((Nx, Ny))

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_t_Ez = np.linspace(0, T, Nt + 1)

# CPML
psi_hxy = np.zeros((Nx, Ny - 1))
psi_hyx = np.zeros((Nx - 1, Ny))
psi_ezx = np.zeros((Nx - 2, Ny - 2))
psi_ezy = np.zeros((Nx - 2, Ny - 2))

sig_x_1D = compute_sigma(Nx, dx, npml, sig_max, m)
sig_y_1D = compute_sigma(Ny, dy, npml, sig_max, m)
sig_x, sig_y = np.meshgrid(sig_x_1D, sig_y_1D, indexing='ij')

b_x = np.exp(-sig_x * dt / eps)
c_x = b_x - 1.0
b_y = np.exp(-sig_y * dt / eps)

```

```

c_y = b_y - 1.0

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(100, 240, 6, dtype=int)
Ez_snapshots = []

# ponta de prova
px, py = Nx // 2 + 50, Ny // 2 + 0
probe = []

for n in range(Nt):
    # atualiza Hx
    dEz_y = (Ez[:, 1:] - Ez[:, :-1]) / dy
    psi_hxy = b_y[:, :-1] * psi_hxy + c_y[:, :-1] * dEz_y
    Hx[:, :-1] -= (dt / mu) * (dEz_y + psi_hxy)

    # atualiza Hy
    dEz_x = (Ez[1:, :] - Ez[:-1, :]) / dx
    psi_hyx = b_x[:-1, :] * psi_hyx + c_x[:-1, :] * dEz_x
    Hy[:-1, :] += (dt / mu) * (dEz_x + psi_hyx)

    # atualiza Ez
    dHy_x = (Hy[1:-1, 1:-1] - Hy[:-2, 1:-1]) / dx
    dHx_y = (Hx[1:-1, 1:-1] - Hx[1:-1, :-2]) / dy
    psi_ezx = b_x[1:-1, 1:-1] * psi_ezx + c_x[1:-1, 1:-1] * dHy_x
    psi_ezy = b_y[1:-1, 1:-1] * psi_ezy + c_y[1:-1, 1:-1] * dHx_y
    Ez[1:-1, 1:-1] += (dt / eps) * ((dHy_x + psi_ezx) - (dHx_y + psi_ezy))

    # fonte
    Ez[Nx//2, Ny//2] += np.exp(-0.5 * ((n * dt - (40 * dt)) / (12 * dt))**2)

    # ponta de prova
    probe.append(Ez[px, py])

    # salva para os snapshots
    if n in n_snapshots:
        Ez_snapshots.append(Ez.copy())

```

ANEXO B – CÓDIGO EXERCÍCIO 9.3

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

def compute_sigma_geometric(N, n_pml, sig_start, g):
    sig = np.zeros(N)
    for i in range(N):
        if i < n_pml:
            depth = n_pml - i
            sig[i] = sig_start * (g ** (depth - 1))
        elif i >= N - n_pml:
            depth = i - (N - n_pml) + 1
            sig[i] = sig_start * (g ** (depth - 1))
    return sig

def compute_sigma(N, ds, n_pml, sigma_max, m_order):
    sig = np.zeros(N)
    for i in range(N):
        dist_left = (n_pml - i) * ds
        dist_right = (i - (N - 1 - n_pml)) * ds
        if dist_left > 0:
            sig[i] = sigma_max * (dist_left / (n_pml * ds))**m_order
        elif dist_right > 0:
            sig[i] = sigma_max * (dist_right / (n_pml * ds))**m_order
    return sig

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
frequency = 600e6 # frequencia da fonte
min_lambda = vp / frequency
Nx = 120
Ny = 60
# dx = dy = 0.1 * min_lambda
dx = dy = 5e-2
dt = 0.9 / (vp * np.sqrt(1/dx**2 + 1/dy**2))
Nt = 200
L = Nx * dx
T = Nt * dt

# --- PML ----
npml = 30
m = 3.75
g_factor = 3
R_err = 1e-6
eta = np.sqrt(mu / eps)
d_pml = npml * dx
sig_max = -(m + 1) * np.log(R_err) / (2 * eta * d_pml)

numerator = -np.log(R_err) * (g_factor - 1)
denominator = 2 * eta * dx * (g_factor**npml - 1)
sig_min = numerator / denominator

# Declara os campos
Ez = np.zeros((Nx, Ny))

```

```

Hx = np.zeros((Nx, Ny))
Hy = np.zeros((Nx, Ny))

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_t_Ez = np.linspace(0, T, Nt + 1)

# CPML
psi_hxy = np.zeros((Nx, Ny - 1))
psi_hyx = np.zeros((Nx - 1, Ny))
psi_ezx = np.zeros((Nx - 2, Ny - 2))
psi_ezy = np.zeros((Nx - 2, Ny - 2))

# polinomial grading
sig_x_1D = compute_sigma(Nx, dx, npml, sig_max, m)
sig_y_1D = compute_sigma(Ny, dy, npml, sig_max, m)
sig_x, sig_y = np.meshgrid(sig_x_1D, sig_y_1D, indexing='ij')

# geometric grading
sig_x_1D = compute_sigma_geometric(Nx, npml, sig_min, g_factor)
sig_y_1D = compute_sigma_geometric(Ny, npml, sig_min, g_factor)
sig_x, sig_y = np.meshgrid(sig_x_1D, sig_y_1D, indexing='ij')

b_x = np.exp(-sig_x * dt / eps)
c_x = b_x - 1.0
b_y = np.exp(-sig_y * dt / eps)
c_y = b_y - 1.0

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(0, Nt - 5, 6, dtype=int)
Ez_snapshots = []

# ponta de prova
px, py = Nx // 2 + 10, Ny // 2 + 10
probe = []

for n in range(Nt):
    # atualiza Hx
    dEz_y = (Ez[:, 1:] - Ez[:, :-1]) / dy
    psi_hxy = b_y[:, :-1] * psi_hxy + c_y[:, :-1] * dEz_y
    Hx[:, :-1] -= (dt / mu) * (dEz_y + psi_hxy)

    # atualiza Hy
    dEz_x = (Ez[1:, :] - Ez[:-1, :]) / dx
    psi_hyx = b_x[:-1, :] * psi_hyx + c_x[:-1, :] * dEz_x
    Hy[:-1, :] += (dt / mu) * (dEz_x + psi_hyx)

    # atualiza Ez
    dHy_x = (Hy[1:-1, 1:-1] - Hy[:-2, 1:-1]) / dx
    dHx_y = (Hx[1:-1, 1:-1] - Hx[1:-1, :-2]) / dy
    psi_ezx = b_x[1:-1, 1:-1] * psi_ezx + c_x[1:-1, 1:-1] * dHy_x
    psi_ezy = b_y[1:-1, 1:-1] * psi_ezy + c_y[1:-1, 1:-1] * dHx_y
    Ez[1:-1, 1:-1] += (dt / eps) * ((dHy_x + psi_ezx) - (dHx_y + psi_ezy))

    # fonte
    Ez[Nx//2, Ny//2] += np.exp(-0.5 * ((n * dt - (40 * dt)) / (12 * dt))**2)
    Ez[Nx//2, Ny//2] += np.sin(2 * np.pi * frequency * n * dt)

    # ponta de prova
    probe.append(Ez[px, py])

```

```
# salva para os snapshots  
if n in n_snapshots:  
    Ez_snapshots.append(Ez.copy())
```

ANEXO C – CÓDIGO EXERCÍCIO 9.4

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
frequency = 600e6 # frequencia da fonte
min_lambda = vp / frequency
Nx = 120
Ny = 60
# dx = dy = 0.1 * min_lambda
dx = dy = 5e-2
dt = 0.9 / (vp * np.sqrt(1/dx**2 + 1/dy**2))
Nt = 200
L = Nx * dx
T = Nt * dt

# Declara os campos
Ez = np.zeros((Nx, Ny))
Hx = np.zeros((Nx, Ny))
Hy = np.zeros((Nx, Ny))

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_t_Ez = np.linspace(0, T, Nt + 1)

# PML
sigma_x = np.zeros((Nx, Ny))
sigma_y = np.zeros((Nx, Ny))
kappa_x = np.ones((Nx, Ny))
kappa_y = np.ones((Nx, Ny))

npml = 30
m = 3.75
sigma_max = -((m+1)*np.log(1e-8))/(2*npml*dx*377)
kappa_max = 8
for i in range(npml):
    sx = sigma_max*((npml-i)/npml)**m
    sigma_x[i,:] = sx
    sigma_x[Nx-1-i,:] = sx
    kappa_x[i,:] = 1 + (kappa_max - 1) * ((npml-i)/npml)**m
    kappa_x[Nx-1-i,:] = 1 + (kappa_max - 1) * ((npml-i)/npml)**m
for j in range(npml):
    sy = sigma_max*((npml-j)/npml)**m
    sigma_y[:,j] = sy
    sigma_y[:,Ny-1-j] = sy
    kappa_y[:,j] = 1 + (kappa_max - 1) * ((npml-j)/npml)**m
    kappa_y[:,Ny-1-j] = 1 + (kappa_max - 1) * ((npml-j)/npml)**m

# Coeficientes da PML
sigma_e = sigma_x + sigma_y
kappa = kappa_x*kappa_y
bx = (1 - ((sigma_x*dt)/(2*eps*kappa_x)))/(1 + ((sigma_x*dt)/(2*eps*kappa_x)))
by = (1 - ((sigma_y*dt)/(2*eps*kappa_y)))/(1 + ((sigma_y*dt)/(2*eps*kappa_y)))

```

```

bz = (1 - ((sigma_e*dt)/(2*eps*kappa)))/(1 + ((sigma_e*dt)/(2*eps*kappa)))
cx = (dt/kappa_x)/(mu*dx*(1 + ((sigma_x*dt)/(2*eps*kappa_x))))
cy = (dt/kappa_y)/(mu*dx*(1 + ((sigma_y*dt)/(2*eps*kappa_y))))
cz = (dt/eps)/(1 + ((sigma_e*dt)/(2*eps*kappa)))

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(0, Nt - 5, 6, dtype=int)
Ez_snapshots = []

# ponta de prova
px, py = Nx // 2 + 10, Ny // 2 + 10
probe = []

for n in range(Nt):
    # atualiza os campos
    Hx[:, :-1] = by[:, :-1]*Hx[:, :-1] - cy[:, :-1]*(Ez[:, 1:] - Ez[:, :-1])
    Hy[:, -1, :] = bx[:, -1, :]*Hy[:, -1, :] + cx[:, -1, :]*(Ez[1:, :] - Ez[:, -1, :])
    Ez[1:-1, 1:-1] = bz[1:-1, 1:-1]*Ez[1:-1, 1:-1] + cz[1:-1, 1:-1]*((Hy[1:-1, 1:-1] - Hy
        [:-2, 1:-1])/dx - (Hx[1:-1, 1:-1] - Hx[1:-1, :-2])/dy)

    # fonte
    Ez[Nx//2, Ny//2] = np.sin(2*np.pi*frequency*n*dt)

    # ponta de prova
    probe.append(Ez[px, py])

    # salva para os snapshots
    if n in n_snapshots:
        Ez_snapshots.append(Ez.copy())

```

ANEXO D – CÓDIGO EXERCÍCIO 9.8

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
Nx = Ny = 200
dx = dy = 1e-3
dt = 0.9 / (vp * np.sqrt(1/dx**2 + 1/dy**2))
Nt = 800
L = Nx * dx
T = Nt * dt

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_t_Ez = np.linspace(0, T, Nt + 1)

def calculate_reflection(probe, start_time):
    return 20 * np.log10(np.max(np.abs(probe[(int(start_time / dt)):])) / np.max(np.abs(
        probe[:]))))

def compute_sigma(N, ds, n_pml, sigma_max, m_order):
    sig = np.zeros(N)
    for i in range(N):
        dist_left = (n_pml - i) * ds
        dist_right = (i - (N - 1 - n_pml)) * ds
        if dist_left > 0:
            sig[i] = sigma_max * (dist_left / (n_pml * ds))**m_order
        elif dist_right > 0:
            sig[i] = sigma_max * (dist_right / (n_pml * ds))**m_order
    return sig

def CPML(spread = 5, px = Nx // 2, py = Ny // 2):
    # Declara os campos
    Ez = np.zeros((Nx, Ny))
    Hx = np.zeros((Nx, Ny))
    Hy = np.zeros((Nx, Ny))

    # --- PML ----
    npml = 30
    m = 3
    R_err = 1e-6
    eta = np.sqrt(mu / eps)
    d_pml = npml * dx
    sig_max = -(m + 1) * np.log(R_err) / (2 * eta * d_pml)

    # CPML
    psi_hxy = np.zeros((Nx, Ny - 1))
    psi_hyx = np.zeros((Nx - 1, Ny))
    psi_ezx = np.zeros((Nx - 2, Ny - 2))
    psi_ezy = np.zeros((Nx - 2, Ny - 2))

    sig_x_1D = compute_sigma(Nx, dx, npml, sig_max, m)
    sig_y_1D = compute_sigma(Ny, dy, npml, sig_max, m)

```

```

sig_x, sig_y = np.meshgrid(sig_x_1D, sig_y_1D, indexing='ij')

b_x = np.exp(-sig_x * dt / eps)
c_x = b_x - 1.0
b_y = np.exp(-sig_y * dt / eps)
c_y = b_y - 1.0

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(100, 240, 6, dtype=int)
Ez_snapshots = []

# ponta de prova
probe = []

for n in range(Nt):
    # atualiza Hx
    dEz_y = (Ez[:, 1:] - Ez[:, :-1]) / dy
    psi_hxy = b_y[:, :-1] * psi_hxy + c_y[:, :-1] * dEz_y
    Hx[:, :-1] -= (dt / mu) * (dEz_y + psi_hxy)

    # atualiza Hy
    dEz_x = (Ez[1:, :] - Ez[:-1, :]) / dx
    psi_hyx = b_x[:-1, :] * psi_hyx + c_x[:-1, :] * dEz_x
    Hy[:-1, :] += (dt / mu) * (dEz_x + psi_hyx)

    # atualiza Ez
    dHy_x = (Hy[1:-1, 1:-1] - Hy[:-2, 1:-1]) / dx
    dHx_y = (Hx[1:-1, 1:-1] - Hx[1:-1, :-2]) / dy
    psi_ezx = b_x[1:-1, 1:-1] * psi_ezx + c_x[1:-1, 1:-1] * dHy_x
    psi_ezy = b_y[1:-1, 1:-1] * psi_ezy + c_y[1:-1, 1:-1] * dHx_y
    Ez[1:-1, 1:-1] += (dt / eps) * ((dHy_x + psi_ezx) - (dHx_y + psi_ezy))

    # fonte
    Ez[Nx//2, Ny//2] += np.exp(-0.5 * ((n * dt - (40 * dt)) / (spread * dt))**2)

    # ponta de prova
    probe.append(Ez[px, py])

return probe

def UPML(spread = 5, px = Nx // 2, py = Ny // 2):
    # Declara os campos
    Ez = np.zeros((Nx, Ny))
    Hx = np.zeros((Nx, Ny))
    Hy = np.zeros((Nx, Ny))

    # PML
    sigma_x = np.zeros((Nx, Ny))
    sigma_y = np.zeros((Nx, Ny))
    kappa_x = np.ones((Nx, Ny))
    kappa_y = np.ones((Nx, Ny))

    npml = 30
    m = 3.75
    sigma_max = -((m+1)*np.log(1e-8))/(2*npml*dx*377)
    kappa_max = 8
    for i in range(npml):
        sx = sigma_max*((npml-i)/npml)**m
        sigma_x[i, :] = sx
        sigma_x[Nx-1-i, :] = sx

```

```

    kappa_x[i,:] = 1 + (kappa_max - 1) * ((npml-i)/npml)**m
    kappa_x[Nx-1-i,:] = 1 + (kappa_max - 1) * ((npml-i)/npml)**m
for j in range(npml):
    sy = sigma_max*((npml-j)/npml)**m
    sigma_y[:,j] = sy
    sigma_y[:,Ny-1-j] = sy
    kappa_y[:,j] = 1 + (kappa_max - 1) * ((npml-j)/npml)**m
    kappa_y[:,Ny-1-j] = 1 + (kappa_max - 1) * ((npml-j)/npml)**m

# Coeficientes da PML
sigma_e = sigma_x + sigma_y
kappa = kappa_x*kappa_y
bx = (1-((sigma_x*dt)/(2*eps*kappa_x)))/(1+((sigma_x*dt)/(2*eps*kappa_x)))
by = (1-((sigma_y*dt)/(2*eps*kappa_y)))/(1+((sigma_y*dt)/(2*eps*kappa_y)))
bz = (1-((sigma_e*dt)/(2*eps*kappa)))/(1+((sigma_e*dt)/(2*eps*kappa)))
cx = (dt/kappa_x)/(mu*dx*(1+((sigma_x*dt)/(2*eps*kappa_x))))
cy = (dt/kappa_y)/(mu*dx*(1+((sigma_y*dt)/(2*eps*kappa_y))))
cz = (dt/eps)/(1+((sigma_e*dt)/(2*eps*kappa)))

# ponta de prova
probe = []

for n in range(Nt):
    # atualiza os campos
    Hx[:, :-1] = by[:, :-1]*Hx[:, :-1] - cy[:, :-1]*(Ez[:, 1:] - Ez[:, :-1])
    Hy[:, -1, :] = bx[:, -1, :]*Hy[:, -1, :] + cx[:, -1, :]*(Ez[1:, :] - Ez[:, -1, :])
    Ez[1:-1, 1:-1] = bz[1:-1, 1:-1]*Ez[1:-1, 1:-1] + cz[1:-1, 1:-1]*((Hy[1:-1, 1:-1] - Hy
       [:, -1, 1:-1])/dx - (Hx[1:-1, 1:-1] - Hx[1:-1, :-2])/dy)

    # fonte
    Ez[Nx//2, Ny//2] += np.exp(-0.5 * ((n * dt - (40 * dt)) / (spread * dt))**2)

    # ponta de prova
    probe.append(Ez[px, py])

return probe

spread_list = [3, 5, 7, 9]

for s in spread_list:
    probe_test = UPML(s, Nx // 2 + 10, Ny // 2 + 10)
    print(calculate_reflection(probe_test, 0.5e-9))

for s in spread_list:
    probe_test = CPML(s, Nx // 2 + 0, Ny // 2 + 10)
    print(calculate_reflection(probe_test, 0.5e-9))

```